Dual-Node I2C Distributed SCADA Mesh Architecture & Safety Report

1. Executive Summary & Problem Statement

Mission-critical industrial IoT edge environments require the separation of environmental monitoring from mechanical actuation to prevent single points of failure. This project implements a multi-node edge system utilizing two interconnected Arduino Uno controllers that communicate via the Inter-Integrated Circuit (I2C) protocol. The architecture successfully isolates volatile sensor acquisition from automated failsafe execution.

2. System Architecture & Topology

The system is divided into two distinct processing nodes operating in a Master-Slave I2C configuration:

- Node A (Sensor Edge Node / I2C Slave - Address 8): Dedicated to environmental data acquisition. It interfaces with an analog Gas Sensor (MQ-2 equivalent) on pin A1, a TMP36 Temperature Sensor on pin A0, and a Photoresistor (LDR) on pin A2. The node continuously reads raw analog values, processes ADC conversions, and packs the telemetry into a compact C++ struct payload.
- Node B (Actuator & SCADA Node / I2C Master): Acts as the central controller and local operator interface. It actively polls Node A for telemetry and manages the local 16×2 I2C LCD, a multi-stage RGB warning LED, a piezo acoustic alarm, and a high-torque ventilation DC motor.

3. I2C Communication Protocol

Data exchange is managed via the standard Wire.h library. Node A serializes the multi-sensor float values (Gas PPM, Temperature Celsius, Lux) into a continuous

binary payload. Node B requests this exact struct size at a fixed 1-second polling interval. This synchronized byte-packing routine ensures deterministic latency and prevents bus starvation during high-frequency environmental sampling.

4. Hazard Mitigation & Automated Failsafes

Node B executes a strict priority-based safety state machine. The environmental hazard threshold is strictly calibrated at 400.0 PPM for volatile gas concentrations.

- Normal Operations (Gas <= 400.0 PPM): The system displays real-time telemetry on the local LCD. The RGB LED is set to solid Green, acoustic alarms remain disabled, and the exhaust mechanism is dormant.
- Emergency Shutdown (Gas > 400.0 PPM): Upon breaching the threshold, Node B overrides the normal state. The LCD flashes a "GAS WARNING!" overlay, the RGB LED transitions to Red, the piezo alarm emits a continuous 1000Hz tone, and the emergency exhaust ventilation motor is activated automatically.

5. Hardware Isolation & Overcurrent Protection

To prevent digital pin burnout during emergency motor activation, the ventilation DC motor is not driven directly by the Arduino GPIO. An NPN transistor (2N2222) is implemented as a solid-state switch on digital Pin 9. The Arduino supplies a low-current base control signal, allowing the motor to draw high-current operational power directly from the 5V power rail, ensuring absolute hardware stability during hazard mitigation sequences.

6. Serialized JSON Edge Telemetry (Output Evidence)

The SCADA gateway (Node B) formats and outputs telemetry data over the serial interface as JSON strings, simulating an industrial edge-to-cloud data pipeline.

```
Node A Slave Initialized. Acquiring real-time data...
{"gasPPM": 306.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
{"gasPPM": 353.00, "tempC": 24.78, "lux": 1023.00}
```

Example Stream:

```
None
{"gasPPM": 420.00, "tempC": 25.50, "lux": 180.00}
```

Appendix A: Node A (Sensor Edge) Source Code

```
None
#include <Wire.h>

#define TEMP_PIN A0
#define GAS_PIN A1
#define LDR_PIN A2

struct SensorData {
  float gasPPM;
  float tempC;
  float lux;
} payload;

void setup() {
  Serial.begin(9600);
  Wire.begin(8);
  Wire.onRequest(requestEvent);
}
```

```
void loop() {
  float rawGas = analogRead(GAS_PIN);
  float rawLux = analogRead(LDR_PIN);
  int rawTemp = analogRead(TEMP_PIN);

  float voltage = rawTemp * (5.0 / 1023.0);
  float tempCelsius = (voltage - 0.5) * 100.0;

  payload.gasPPM = rawGas;
  payload.lux = rawLux;
  payload.tempC = tempCelsius;

  delay(500);
}

void requestEvent() {
  Wire.write((byte*)&payload, sizeof(payload));
}
```

Appendix B: Node B (SCADA Actuator) Source Code

None

```
#include <Wire.h>
#include <Adafruit_LiquidCrystal.h>

Adafruit_LiquidCrystal lcd(0);

#define RED_PIN 11
#define BLUE_PIN 10
#define GREEN_PIN 6
#define BUZZER_PIN 12
#define MOTOR_PIN 9

struct SensorData {
  float gasPPM;
```

```
  float tempC;
  float lux;
} payload;

void setup() {
  Serial.begin(9600);
  Wire.begin();

  pinMode(RED_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(MOTOR_PIN, OUTPUT);

  lcd.begin(16, 2);
  lcd.print("System Ready");
  delay(1000);
}

void loop() {
  Wire.requestFrom(8, sizeof(payload));

  if (Wire.available() == sizeof(payload)) {
    Wire.readBytes((byte*)&payload, sizeof(payload));

    // JSON Formatted Serial Telemetry
    Serial.print("{\"gasPPM\": ");
    Serial.print(payload.gasPPM);
    Serial.print(", \"tempC\": ");
    Serial.print(payload.tempC);
    Serial.print(", \"lux\": ");
    Serial.print(payload.lux);
    Serial.println("}");

    lcd.clear();

    if (payload.gasPPM > 400.0) {
      // DANGER STATE
      lcd.setCursor(0, 0);
      lcd.print("GAS WARNING!");
```

```
      lcd.setCursor(0, 1);
      lcd.print("PPM: ");
      lcd.print(payload.gasPPM);

      analogWrite(RED_PIN, 255);
      analogWrite(BLUE_PIN, 0);
      analogWrite(GREEN_PIN, 0);
      tone(BUZZER_PIN, 1000);
      digitalWrite(MOTOR_PIN, HIGH);

    } else {
      // NORMAL STATE
      lcd.setCursor(0, 0);
      lcd.print("Gas: ");
      lcd.print(payload.gasPPM);
      lcd.setCursor(0, 1);
      lcd.print("Temp: ");
      lcd.print(payload.tempC);
      lcd.print(" C");

      analogWrite(RED_PIN, 0);
      analogWrite(BLUE_PIN, 0);
      analogWrite(GREEN_PIN, 255);
      noTone(BUZZER_PIN);
      digitalWrite(MOTOR_PIN, LOW);
    }
  }
  delay(1000);
}
```